Ex. No. 12	BUILD WEB APP WITH FLUTTER
Date of Exercise	30/09/2026

Aim

To develop and build a Flutter application for the Web.

Description:

1. Introduction

Flutter is a cross-platform framework from Google. From one Dart codebase it can produce apps for Android, iOS, web, Windows, macOS and Linux. Flutter Web compiles the Dart code to JavaScript, which any modern browser can run. That means a Flutter app can be published as an ordinary website without writing HTML, CSS or JavaScript by hand.

This experiment has three goals:

1. Run a Flutter app in a browser.
2. Make the interface responsive, so it adapts to the screen size.
3. Produce a production build that can be deployed to a web server.

2. Tools Used

- Flutter SDK creates, runs and builds the project.
- Dart is the programming language.
- Google Chrome is the browser used to run and test the app. Flutter treats it as a device.
- VS Code is the editor, opened with code ..
- Hot reload shows code changes in the browser without a full restart.

3. Explanation of Each Step

Step 1: flutter devices

This lists every target Flutter can run on. For web you should see Chrome (web) and usually Web Server (web). If neither appears, web support may be off, and flutter config --enable-web turns it on.

Step 2: flutter create flutter_web_app, cd flutter_web_app, code .

flutter create generates the full project. The important parts are:

- lib/main.dart, which holds the Dart code you edit.
- web/, which holds index.html, manifest.json and the icons. This folder is what makes the project a web target.
- pubspec.yaml, which lists dependencies and assets.

cd enters the folder, and code . opens it in VS Code.

Step 3: flutter run -d chrome

The -d chrome flag selects Chrome as the device. Flutter compiles the app in debug mode and opens it at an address like localhost:52814. In the terminal, r hot reloads, R hot restarts and q quits.

Step 4: Create the web interface

The code uses three widgets:

- Scaffold is the basic page structure. It provides the app bar and the body area.
- AppBar is the blue title bar at the top, showing "Flutter Web Application".
- Center with ElevatedButton places a raised button labelled "Welcome" in the middle of the page. onPressed: () {} is an empty handler, so the button looks and responds like a button but does nothing yet.

Step 5: Make it responsive

LayoutBuilder gives the code the space the widget has available, as constraints. The check is simple:

- If constraints.maxWidth < 600, the screen is narrow (a phone or a small window), so it shows "Mobile Web Layout".
- Otherwise the screen is wide, so it shows "Desktop Web Layout".

The 600 px breakpoint is a common choice. Because the width is read each time the layout is built, the screen updates as the user drags the window edge.

Step 6: Test the browser window

Drag the Chrome window narrower and wider. The text should switch at about 600 px. This confirms the layout responds to the screen width. You can also open Chrome DevTools (F12) and use device mode to test phone sizes.

Step 7: flutter build web

This creates the release (production) build. The build:

- Compiles Dart to optimised, minified JavaScript (main.dart.js).
- Removes debug tools so the app is smaller and faster.
- Copies the web assets and icons.

Run it from the project folder. It takes noticeably longer than a debug run.

Step 8: Check build/web/

This folder is the finished website. Typical contents:

File or folder	Purpose
index.html	Entry page that loads the app
main.dart.js	The compiled Dart app
flutter.js, flutter_bootstrap.js	Loads and starts the Flutter engine
flutter_service_worker.js	Caching for faster reloads
assets/	Fonts and images
canvaskit/	Rendering engine files
icons/, favicon.png	App and tab icons

4. Explanation of the Program

- main() calls runApp(const MyApp()), which starts the app.
- MyApp is a StatelessWidget that returns a MaterialApp. It sets the title, hides the debug banner, applies a blue Material 3 theme and sets HomePage as the first screen.
- HomePage returns a Scaffold. Its body is a LayoutBuilder, so the page can react to the available width.
- Inside the builder, layoutText stores either "Mobile Web Layout" or "Desktop Web Layout" using a conditional expression (condition ? a : b).
- A Column stacks the text and the button, and mainAxisAlignment: center keeps them in the middle. A SizedBox adds spacing between them.

  The app uses a StatelessWidget because nothing changes through user interaction. The only thing that changes is the window size, which LayoutBuilder handles by itself.

  5. Observations
- The app runs in Chrome with the same look as on mobile, from one codebase.
- The layout text switches at the 600 px breakpoint when the window is resized.
- flutter build web produces a deployable folder in build/web/.

PROCEDURE

1. Check that Flutter Web is available: flutter devices. Look for Chrome or another configured web browser in the list.
2. Create the project:
3. Run the app on Chrome: flutter run -d chrome
4. Create the web interface in lib/main.dart using a Scaffold with an AppBar and a centred ElevatedButton.
5. Make it responsive with a LayoutBuilder that checks constraints.maxWidth and shows a mobile or desktop layout.
6. Test in the browser. Resize the Chrome window and verify that the layout changes with the screen width.
7. Create the production build: flutter build web
8. Check the generated files under build/web/.

PROGRAM

```
    import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
    title: 'Flutter Web Application',
    debugShowCheckedModeBanner: false,

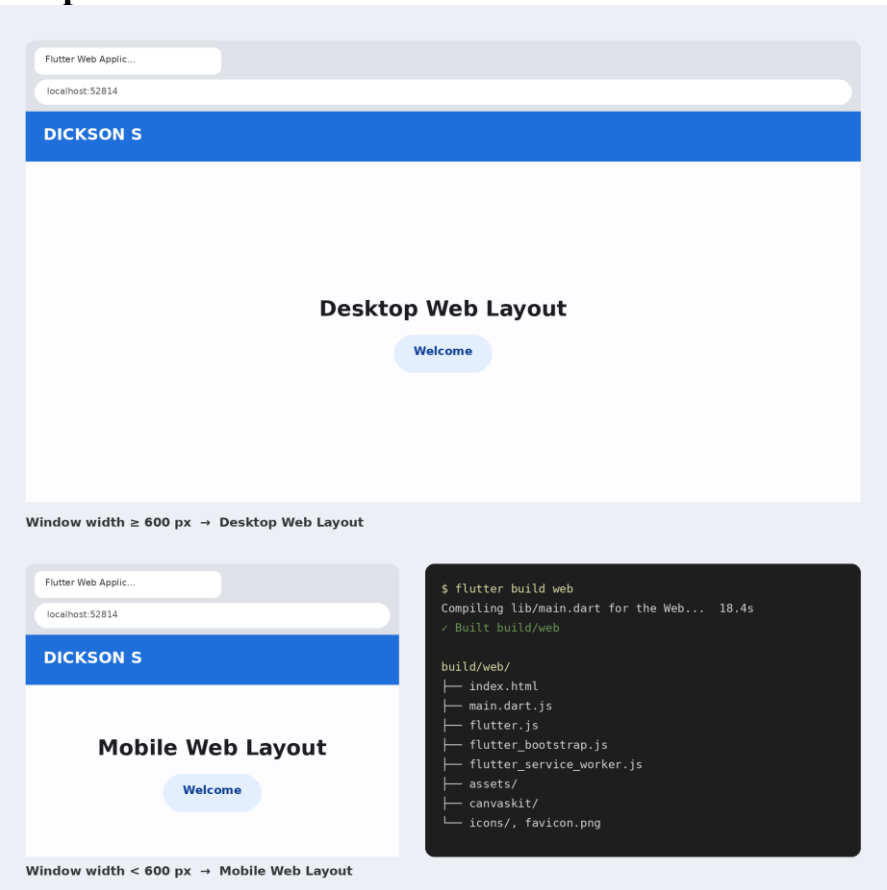
    theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
    home: const HomePage(),
   );
  }
}

class HomePage extends StatelessWidget {
 const HomePage({super.key});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: const Text('Flutter Web Application'),
   ),
   body: LayoutBuilder(
    builder: (context, constraints) {
     // Responsive text based on browser window width
     final String layoutText = constraints.maxWidth < 600
        ? 'Mobile Web Layout'
        : 'Desktop Web Layout';

     return Center(
      child: Column(
       mainAxisAlignment: MainAxisAlignment.center,
       children: [
        Text(
         layoutText,
         style: const TextStyle(
            fontSize: 24, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 20),
        ElevatedButton(
         onPressed: () {},
         child: const Text('Welcome'),
        ),
       ],
      ),
     );
    },
   ),
  );
```

Output



Result:

A responsive Flutter application was successfully developed, executed on Chrome and built for the Web. When the browser window is narrower than 600 px it shows "Mobile Web Layout", and at 600 px or wider it shows "Desktop Web Layout". flutter build web generated the production files under build/web/.